

Processing and analysis of nanopore sequences

Marko Suokas

Preprocess reads

Dorado does not support demultiplexing dual indexes located on both the 5' and 3' ends. Additionally, in ligated libraries, the reads can appear in either orientation. To address this, we use cutadapt for demultiplexing. Index pairs are identified using the linked adapters approach in both forward and reverse orientations, after which scripts are applied to reverse complement the reverse reads. Finally, the reads are merged.

Extract Forward Reads

You can extract forward reads using the following ion torrent adapter sequences with following command:

```
cutadapt --trimmed-only -e 0.05 -o reanalysis/fwd.fastq.gz \  
-g "CCATCTCATCCCTGCGTGTCTCCGACTCAG;o=30...ATCACCGACTGCCCATAGAGAGG;o=23" \  
evseq_rebasecall.fastq.gz
```

Extract Reverse Reads

To extract reverse reads, use the reverse-complement of adapter sequences

```
cutadapt --trimmed-only -e 0.05 -o reanalysis/reverse.fastq.gz \  
-g "CCTCTCTATGGGCAGTCGGTGAT;o=23...CTGAGTCGGAGACACGCAGGGATGAGATGG;o=30" \  
evseq_rebasecall.fastq.gz
```

Reverse Complement Reverse Reads

```
seqkit seq -rp --seq-type DNA -o revcomp.fastq.gz \  
reverse.fastq.gz
```

Merge Forward and Reverse Reads

For the final step, you can merge forward and reverse reads with the same base name from two directories. Here's a simple bash command for that:

```
cat forward.fastq.gz rev.fastq.gz >raw.fastq.gz
```

Read quality

Nanopore sequencing Phred scores should be interpreted with caution, as per-base accuracy cannot be directly determined from the electrical signal. Instead, scores are estimates based on the confidence of the basecalling model rather than a direct probability of sequencing error. As a result, traditional quality filtering approaches may not always be appropriate.

ONT instead calculates an estimated cumulative error rate and converts it into an average quality score per read. The following code computes this metric and generates a violin plot showing the distribution of average sequence quality per read for each sample.

In this set we have 132 samples. We will analyze quality of only 48 samples in order to keep total number in plot manageable.

```
# Define path
source_dir <- "data/reads/"
# A function to calculate so called AvgQual
extract_nano_qscores <- function(file) {
  # Read fastq
  fq <- readFastq(file)
  # Extract q-values to a matrix
  qmat <- as(quality(fq), "matrix")
  # Convert values to error probabilities
  error_probs <- 10^(-qmat/10)
  # Compute number of expected errors per read
  total_errors <- rowSums(error_probs, na.rm = T)
  # Length per read
  read_lengths <- rowSums(!is.na(qmat))
  # Compute "ONT-style" Q-score
  ont_qscores <- -10*log10(total_errors/read_lengths)
  # Trim file names
  sampleid <- sub("_.*|\\.fastq\\.gz$", "", basename(file))
  # Create a dataframe
  qdata <- tibble(
    nanopore_qscore = ont_qscores,
    sampleid = sampleid)
  return(qdata)
}

# Create list of files
```

```

files <- list.files(source_dir, pattern = "\\..fastq\\.gz$",
                    full.names = T)
# Subset first 48 samples
files <- files[1:48]
# Apply function and use map_dfr to combine dataframes together
qscore_data <- map_dfr(files, extract_nano_qscores)
# Save result to a rds file
saveRDS(qscore_data, "data/nanopore_quality.rds")

```

Violin plot

```

# Read data
qdata <- readRDS("data/nanopore_quality.rds")

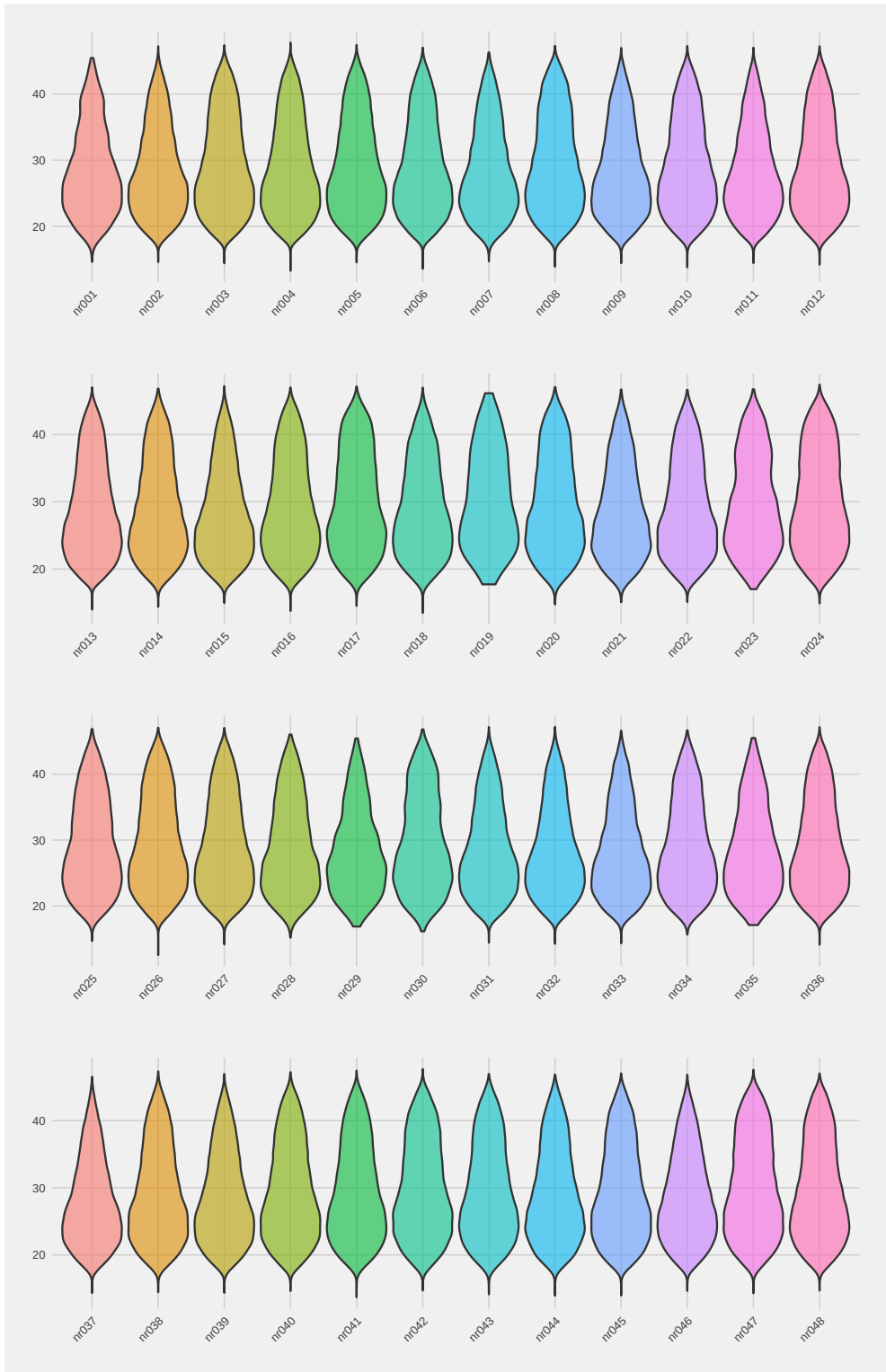
# Divide samples to groups of 12
sample_list <- unique(qdata$sampleid)
sample_groups <- split(sample_list, ceiling(seq_along(sample_list) / 12))

# Generate a separate plot for each group
plots <- map(sample_groups, function(group_samples) {
  ggplot(filter(qdata, sampleid %in% group_samples),
          aes(x = sampleid, y = nanopore_qscore, fill = sampleid)) +
    geom_violin(scale = "width", alpha = 0.6) +
    theme_fivethirtyeight(base_size=8) +
    labs(x = "Sample", y = "Nanopore Q-score") +
    theme(legend.position = "none", axis.text.x = element_text(angle = 45, hjust = 1))
})

# Combine all plots into a single vertical layout
final_plot <- wrap_plots(plots, ncol = 1)

# Show the plot
print(final_plot)

```



Import sequence data to Qiime 2

The create_manifest.sh script inside root directory can be used for creation of manifest file needed to import demuxed and trimmed sequence files.

```
# Activate qiime environment
export PATH="/homedir04/msuokas/miniconda3/bin:$PATH"
source activate qiime-2024.10

# Import sequence data
qiime tools import \
  --type 'SampleData[SequencesWithQuality]' \
  --input-path data/reads/manifest.csv \
  --output-path data/demux.qza \
  --input-format SingleEndFastqManifestPhred33
```

Dereplicate sequences

Removes sequence redundancy.

```
# Activate qiime environment
export PATH="/homedir04/msuokas/miniconda3/bin:$PATH"
source activate qiime-2024.10

# Dereplicate sequences with vsearch plugin
qiime vsearch dereplicate-sequences \
  --o-dereplicated-table data/derep_table.qza \
  --o-dereplicated-sequences data/derep_seqs.qza \
  --i-sequences data/demux.qza
```

Pick de-novo otus

Clustering of sequences at 97 % identity level

```
# Activate qiime environment
export PATH="/homedir04/msuokas/miniconda3/bin:$PATH"
source activate qiime-2024.10

# Cluster
qiime vsearch cluster-features-de-novo \
  --i-sequences data/derep_seqs.qza \
  --i-table data/derep_table.qza \
  --p-strand plus --p-threads 10 --p-perc-identity 0.97 \
  --o-clustered-table data/clustered-table.qza \
  --o-clustered-sequences data/clustered-seqs.qza
```

Filter rare features

```
# Activate qiime environment
export PATH="/homedir04/msuokas/miniconda3/bin:$PATH"
source activate qiime-2024.10

# Filter table
qiime feature-table filter-features \
--i-table data/clustered-table.qza \
--p-min-frequency 10 \
--o-filtered-table data/n10-table.qza

# Filter sequences
qiime feature-table filter-seqs \
--i-data data/clustered-seqs.qza \
--i-table data/n10-table.qza \
--o-filtered-data data/n10-seqs.qza
```

Detect and remove chimeric features

```
# Activate qiime environment
export PATH="/homedir04/msuokas/miniconda3/bin:$PATH"
source activate qiime-2024.10

# Detect chimeric features
qiime vsearch uchime-denovo \
--i-table data/n10-table.qza \
--i-sequences data/n10-seqs.qza \
--o-nonchimeras data/nonchimeric.qza \
--o-stats data/stats.qza \
--o-chimeras data/chimeric.qza

# Clean table
qiime feature-table filter-features \
--i-table data/n10-table.qza \
--m-metadata-file data/nonchimeric.qza \
--o-filtered-table data/feature-table.qza

# Clean sequences
qiime feature-table filter-seqs \
--i-data data/n10-seqs.qza \
--i-table data/feature-table.qza \
--o-filtered-data data/feature-seqs.qza
```

Import feature table and metadata to R

```
#sequence file
tse <- importQIIME2(featureTableFile = "data/feature-table.qza")
tse <- tse[, sort(colnames(tse))]
#add metadata
metadata <- data.frame(read_tsv("data/jt_meta.tsv",
                               show_col_types = F))
metadata <- column_to_rownames(metadata, "SampleID")
colData(tse) <- DataFrame(metadata)
tse
```

```
class: TreeSummarizedExperiment
dim: 7207 131
metadata(0):
assays(1): counts
rownames(7207): efff17dee73a28bd87be7b58eb012da9f68fd1ff
                2a7d89c5f313d7488ceb4d8d99f88de12f28221a ...
                65aace9aa5ae0e5eb076acccde664987d8478f69
                40d4c507e5b062fc94af72bb6a2c46148a1f98d0
rowData names(0):
colnames(131): nr001 nr002 ... nr131 nr132
colData names(4): Name Source Type Barcode_seq
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: NULL
rowTree: NULL
colLinks: NULL
colTree: NULL
```

Assign taxonomy

Confirm correct sequence order first as QIIME shuffles natural order.

```
ref_sequences <- importQZA("data/feature-seqs.qza")
ref_ids <- names(ref_sequences)
tse_ids <- rownames(tse)
# Check if all rownames are present in the reference IDs
if (!all(tse_ids %in% ref_ids)) {
  stop("Not all rownames from tse are present in the reference sequences.")
}
# Reorder `ref_sequences` to match the order of `tse` rownames
ref_sequences_ordered <- ref_sequences[match(tse_ids, ref_ids)]
all(names(ref_sequences_ordered) == rownames(tse))
```

```
[1] TRUE
```

```
referenceSeq(tse) <- ref_sequences_ordered
tse

class: TreeSummarizedExperiment
dim: 7207 131
metadata(0):
assays(1): counts
rownames(7207): efff17dee73a28bd87be7b58eb012da9f68fd1ff
                2a7d89c5f313d7488ceb4d8d99f88de12f28221a ...
                65aace9aa5ae0e5eb076accde664987d8478f69
                40d4c507e5b062fc94af72bb6a2c46148a1f98d0
rowData names(0):
colnames(131): nr001 nr002 ... nr131 nr132
colData names(4): Name Source Type Barcode_seq
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: NULL
rowTree: NULL
colLinks: NULL
colTree: NULL
referenceSeq: a DNASTringSet (7207 sequences)
```

Assign taxonomy using Silva reference database.

```
# assign taxa
taxa <- assignTaxonomy(referenceSeq(tse), minBoot=90, multithread=10,
                        refFasta=~/reference/silva_nr99_v138.1_train_set.fa.gz")
# add species column (empty)
taxa <- data.frame(taxa)
taxa$species <- "NA"
saveRDS(taxa, "data/taxa.rds" )
```

Include taxonomy table

```
# Read taxa file
taxa <- readRDS("data/taxa.rds")
# Add to rowData
rowData(tse) <- taxa
# Rename rows
rownames(tse) <- paste0("OTU", seq(1,nrow(tse)))
# Check tse
tse
```

```

class: TreeSummarizedExperiment
dim: 7207 131
metadata(0):
assays(1): counts
rownames(7207): OTU1 OTU2 ... OTU7206 OTU7207
rowData names(7): Kingdom Phylum ... Genus species
colnames(131): nr001 nr002 ... nr131 nr132
colData names(4): Name Source Type Barcode_seq
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: NULL
rowTree: NULL
colLinks: NULL
colTree: NULL
referenceSeq: a DNASTringSet (7207 sequences)

```

Write object data to files

Write RDS. File can be reloaded as an unchanged object in R

```
saveRDS(tse, "results/tse.rds")
```

Write abundance table

```

# FeatureID will be rowname
abd <- data.frame(FeatureID = rownames(tse), assays(tse)$counts)
# Write
write_tsv(abd, "results/feature_table.tsv")

```

Write taxonomy table

```

# FeatureID will be rowname
taxt <- data.frame(FeatureID = rownames(tse), rowData(tse))
# Write
write_tsv(taxt, "results/taxonomy.tsv")

```

Feature sequences to fasta file

```

writeXStringSet(referenceSeq(tse), "results/repseq.fasta",
                 append = F, compress = F,
                 format = "fasta")

```

Write metadata file

```
metadf <- data.frame(colData(tse)) %>% rownames_to_column(var="Sampleid")
# Write
write_tsv(metadf, "results/metadata.tsv")
```

Microbe data analysis

Agglomeration of taxonomy to genus rank and count relative abundance

```
# Agglomeration
altExp(tse, "Genus") <- agglomerateByRank(tse, rank = "Genus",
                                          onRankOnly = T, na.rm = F)
# Calculate relabundance
altExp(tse, "Genus") <- transformAssay(altExp(tse, "Genus"),
                                       assay.type = "counts",
                                       method = "relabundance")
```

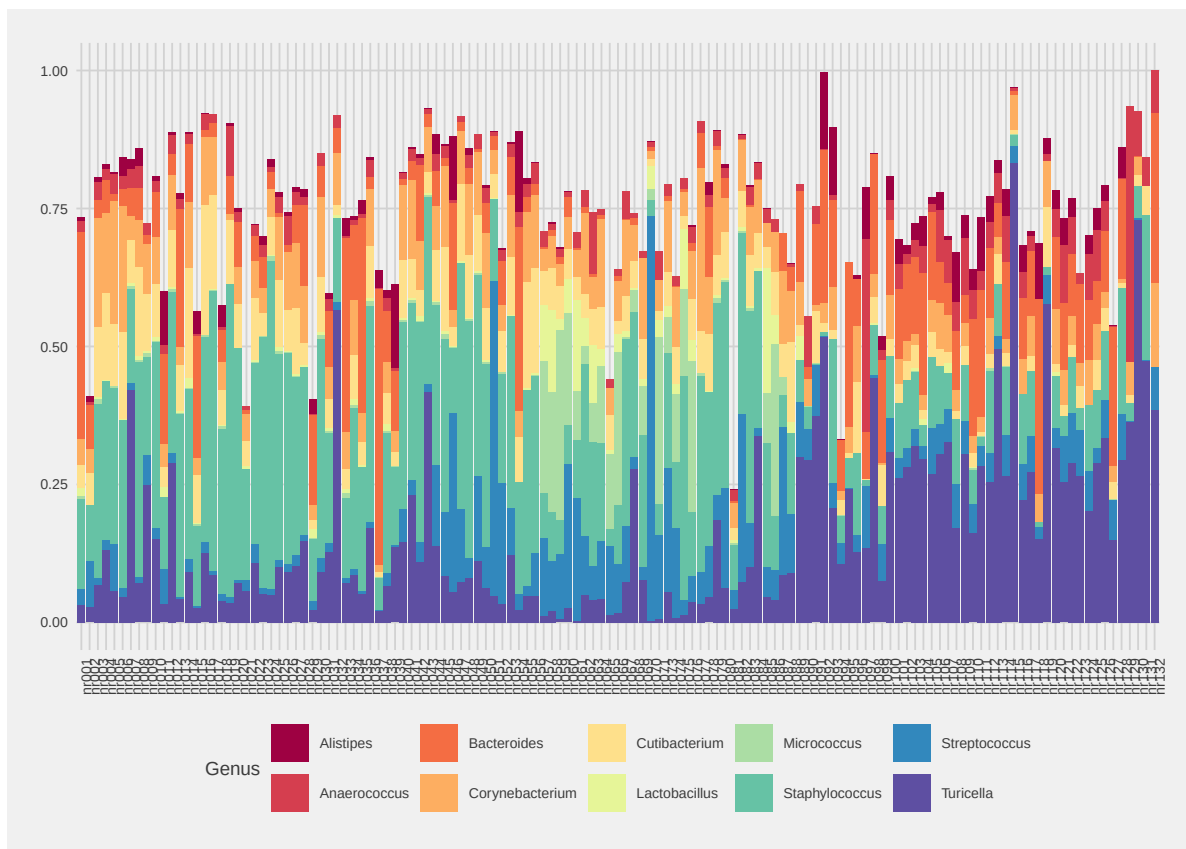
Pick ten most abundant features

```
#top10 features
top10 <- getTop(altExp(tse, "Genus"), top = 10,
               method = "mean",
               assay.type = "relabundance")
#create and filter table
table <- data.frame(assays(altExp(tse, "Genus"))$relabundance)
table <- table %>% rownames_to_column(var = "Genus") %>%
  filter(Genus %in% top10)
```

Composition plot

Change data to long table format

```
df_long <- table %>% pivot_longer(cols = starts_with("nr"),
                                names_to = "Sample", values_to = "Abundance")
#Plot stacked barplot
ggplot(df_long, aes(x = Sample, y = Abundance, fill = Genus)) +
  geom_bar(stat = "identity") +
  theme_fivethirtyeight(base_size = 8) +
  scale_fill_brewer(palette = "Spectral") +
  theme(axis.text.x = element_text(angle = 90))
```



Summary of otu picking strategy

Create summary table

```
summary <- data.frame(Sample = colnames(tse),
                      Counts = colSums(assays(tse)$counts),
                      Features = colSums(assays(tse)$counts != 0))

rownames(summary) <- NULL
kable(summary, digits=0) %>%
  kable_styling(latex_options = c("HOLD_position", "striped",
                                   "repeat_header"),
                font_size = 8) %>%
  row_spec(0, background = "teal", color = "ivory")
```

Sample	Counts	Features
nr001	4406	312
nr002	99029	1295

(continued)

Sample	Counts	Features
nr003	136139	1399
nr004	251477	1859
nr005	102400	1414
nr006	122300	1396
nr007	15968	518
nr008	59730	1063
nr009	56144	874
nr010	57762	900
nr011	56359	930
nr012	61975	1051
nr013	178537	1398
nr014	36317	755
nr015	51624	976
nr016	56720	847
nr017	31634	808
nr018	31599	630
nr019	203	34
nr020	16902	752
nr021	12502	692
nr022	36019	879
nr023	5560	386
nr024	38068	897
nr025	17771	597
nr026	52666	1320
nr027	55963	989
nr028	6113	428
nr029	1094	141
nr030	2852	260
nr031	53460	972
nr032	52247	829
nr033	39364	1014
nr034	24270	775
nr035	953	119
nr036	54457	1014
nr037	50196	1105
nr038	61724	1237
nr039	60700	1124
nr040	64700	1145
nr041	49016	933
nr042	55057	918
nr043	61744	915
nr044	50770	988
nr045	61379	1232
nr046	56533	668
nr047	57827	834
nr048	54901	954
nr049	58008	855
nr050	57415	1119
nr051	57034	753
nr052	58022	1214

(continued)

Sample	Counts	Features
nr053	51112	817
nr054	52586	1027
nr055	43736	1025
nr056	55299	1077
nr057	55631	757
nr058	28477	327
nr059	48515	530
nr060	55007	652
nr061	38580	585
nr062	56382	548
nr063	40491	493
nr064	56297	722
nr065	30449	322
nr066	45865	499
nr067	27380	360
nr068	29320	390
nr069	29309	458
nr070	59358	486
nr071	51978	558
nr073	48891	667
nr074	56280	513
nr075	55357	585
nr076	48799	749
nr077	52710	847
nr078	48074	1139
nr079	992	108
nr080	75124	1127
nr081	72762	1611
nr082	38330	599
nr083	32907	663
nr084	6728	309
nr085	62152	672
nr086	27526	509
nr087	1506	83
nr088	348	48
nr089	91930	683
nr090	45133	318
nr091	42445	385
nr092	85914	496
nr093	166193	874
nr094	179796	603
nr095	184834	578
nr096	149774	561
nr097	148681	470
nr098	60278	417
nr099	6099	144
nr100	176	53
nr101	116	42
nr102	61	31
nr103	178	61

(continued)

Sample	Counts	Features
nr104	188	58
nr105	169	54
nr106	103	47
nr107	169	57
nr108	85	44
nr109	134	51
nr110	423	103
nr111	119	45
nr112	203	57
nr113	136	45
nr114	125	45
nr115	493	34
nr116	72	42
nr117	118	47
nr118	104	41
nr119	163877	837
nr120	151	52
nr121	155	58
nr122	189	63
nr123	58	32
nr124	100	46
nr125	81	35
nr126	80	31
nr127	200190	756
nr128	71013	385
nr129	44580	414
nr130	182263	823
nr131	19	7
nr132	14	10

Alpha diversity

```
tse <- addAlpha(tse, assay.type="counts", index="shannon_diversity",
               sample=10000)
shannon <- data.frame(Samples = colnames(tse),
                     Shannon_index = colData(tse)$shannon_diversity)
rownames(shannon) <- NULL

# Table
kable(shannon, digits=2, caption = "Shannon index") %>%
kable_styling(latex_options = c("HOLD_position", "striped",
                                "repeat_header"),
              font_size = 8) %>% row_spec(0, background = "teal",
                                color = "ivory")
```

Table 2: Shannon index

Samples	Shannon_index
nr001	3.44
nr002	4.23
nr003	3.44
nr004	3.25
nr005	3.38
nr006	3.36
nr007	3.04
nr008	3.10
nr009	3.61
nr010	3.21
nr011	4.16
nr012	2.93
nr013	3.37
nr014	2.89
nr015	3.92
nr016	2.59
nr017	2.41
nr018	3.23
nr019	1.97
nr020	3.53
nr021	4.45
nr022	3.76
nr023	3.56
nr024	2.76
nr025	3.18
nr026	3.74
nr027	3.47
nr028	3.54
nr029	3.91
nr030	3.02
nr031	4.18
nr032	2.20
nr033	3.88
nr034	3.63
nr035	3.27
nr036	3.25
nr037	3.84
nr038	3.97
nr039	4.16
nr040	3.25
nr041	3.02
nr042	3.18
nr043	2.33
nr044	3.32
nr045	3.24
nr046	3.05
nr047	2.53
nr048	3.12
nr049	2.99

Table 2: Shannon index (*continued*)

Samples	Shannon_index
nr050	3.45
nr051	2.29
nr052	3.80
nr053	3.10
nr054	3.59
nr055	3.29
nr056	3.18
nr057	3.52
nr058	2.62
nr059	3.32
nr060	3.51
nr061	3.68
nr062	3.27
nr063	3.75
nr064	3.62
nr065	3.30
nr066	3.47
nr067	3.10
nr068	2.81
nr069	3.48
nr070	2.02
nr071	3.35
nr073	3.47
nr074	3.24
nr075	2.88
nr076	3.61
nr077	2.88
nr078	3.67
nr079	2.58
nr080	3.12
nr081	6.04
nr082	2.60
nr083	3.20
nr084	2.65
nr085	3.26
nr086	3.38
nr087	3.20
nr088	3.13
nr089	2.74
nr090	2.56
nr091	2.39
nr092	1.60
nr093	2.84
nr094	3.19
nr095	2.58
nr096	2.43
nr097	2.49
nr098	2.29

Table 2: Shannon index (*continued*)

Samples	Shannon_index
nr099	2.64
nr100	3.17
nr101	3.17
nr102	2.93
nr103	3.23
nr104	3.24
nr105	3.24
nr106	3.15
nr107	3.23
nr108	3.46
nr109	3.19
nr110	3.90
nr111	3.04
nr112	3.22
nr113	2.56
nr114	3.11
nr115	1.27
nr116	3.33
nr117	3.19
nr118	3.20
nr119	2.06
nr120	3.14
nr121	3.46
nr122	3.30
nr123	3.07
nr124	3.41
nr125	3.02
nr126	2.82
nr127	2.99
nr128	2.32
nr129	1.75
nr130	1.51
nr131	1.48
nr132	2.06

Beta diversity measured by Bray-Curtis dissimilarity

```
tse <- runMDS(tse, FUN = getDissimilarity, method = "bray",
              assay.type = "counts", name = "MDS_bray", sample=10000)
#Explained variance
e <- attr(reducedDim(tse, "MDS_bray"), "eig")
rel_eig <- e / sum(e[e > 0])
#Plot
plotReducedDim(tse, "MDS_bray", colour_by = "Source",
               text_by = "Type", text_size = 3, shape_by = "Type") +
  labs(x = paste("PCoA 1 (", round(100 * rel_eig[[1]], 1),
                "%", ")"), sep = ""), y = paste("PCoA 2 (",
                round(100 * rel_eig[[2]], 1),
                "%", ")"), sep = "")) + scale_color_lancet()
```

